Day 148 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 2 reference

Watch: What is Stationarity (Aric LaBarr) | Time Series Talk: Augmented Dickey Fuller Test + Code (ritvikmath)

`day148_stationarity_differencing.py`, segment by segment, in the order `main()` uses things. The autocorrelation function, differencing, the Dickey-Fuller statistic and its simulated table, the ADF test and the KPSS test are new, and are copied unchanged into today's exercise file. Day 147's loaders are imported from its exercise file. statsmodels supplies the reference versions and MacKinnon's published tables. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: y = series · Δy = change · γ = level coefficient · p = lagged changes · ϕ = AR(1) coefficient

Segment 0 — Setup

Imports and constants

CODE

```
from statsmodels.tsa.adfvalues import mackinnoncrit, mackinnonp
from statsmodels.tsa.stattools import acf as sm_acf, adfuller, kpss as sm_kpss
warnings.filterwarnings("ignore")
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day147_exercise as d147
SIMS = 20000
POWER_SIMS = 2000
```

- `from statsmodels.tsa.stattools import acf as sm_acf, ...` — 'as' renames on import, so the library's acf does not clash with ours.
- `with contextlib.redirect_stdout(io.StringIO()):` — everything printed inside the block goes into a string that is thrown away.
- `import day147_exercise as d147` — yesterday's exercise file, imported as a module; its parts are used as d147.load_co2().
- `SIMS = 20000` — a module-level constant; the functions below use it as a default.

Segment 1 — The parts

acf and difference

MATH

$$r_k = \Sigma\,(y_t - m)(y_{t+k} - m) \,/\, \Sigma\,(y_t - m)^2 \quad (m = \text{the mean})$$

CODE

```
def acf(x, nlags):
    x = np.asarray(x, float) - np.mean(x); d = np.sum(x * x)
    return np.array([1.0] + [np.sum(x[:-k] * x[k:]) / d for k in range(1, nlags + 1)])
def difference(y, lag=1):
    return y[lag:] - y[:-lag]
```

- `np.asarray(x, float) - np.mean(x)` — accepts a list or an array, as floats, and centres it.

- `x[:-k] * x[k:]` — two slices k apart: each value times the value k steps later.
- `[1.0] + [... for k in range(1, nlags + 1)]` — joining two Python lists: lag 0 is always 1.
- `y[lag:] - y[:-lag]` — each value minus the one lag places before; the result is lag values shorter.

df_stats: one statistic per column

MATH

$\Delta y_t = \gamma y_{t-1} + c + e_t; \quad \text{statistic} = \gamma / se(\gamma); \quad \text{a random walk has } \gamma = 0$

CODE

```
def df_stats(Y, regression="c"):
    n = len(Y) - 1
    D = np.diff(Y, axis=0); L = Y[:-1]
    det = np.ones((n, 1)) if regression == "c" else np.column_stack([np.ones(n), np.arange(1, n +
        1)])
    Q, _ = np.linalg.qr(det)
    D = D - Q @ (Q.T @ D); L = L - Q @ (Q.T @ L)
    g = np.sum(L * D, axis=0) / np.sum(L * L, axis=0)
    s2 = np.sum((D - g * L) ** 2, axis=0) / (n - det.shape[1] - 1)
    return g / np.sqrt(s2 / np.sum(L * L, axis=0))
```

- `np.diff(Y, axis=0)` — differences down each column: every column is one series.
- `Q, _ = np.linalg.qr(det)` — a QR factorisation; Q's columns span the constant (and trend), and _ discards R.
- `D - Q @ (Q.T @ D)` — removes the constant (and trend) from every column at once, so only one coefficient is left to fit.
- `np.sum(L * D, axis=0) / np.sum(L * L, axis=0)` — one least-squares slope per column, with no loop.
- `det.shape[1]` — how many deterministic terms: 1 or 2, used in the degrees of freedom.

df_table and mc_pvalue

CODE

```
def df_table(n, regression="c", sims=SIMS, seed=0):
    Y = np.cumsum(np.random.RandomState(seed).standard_normal((n, sims)), axis=0)
    return np.sort(df_stats(Y, regression))
def mc_pvalue(stat, table):
    return np.searchsorted(table, stat, side="right") / len(table)
```

- `standard_normal((n, sims))` — an n by sims matrix of noise; cumsum(axis=0) turns each column into a random walk.
- `np.sort(...)` — sorted once, so a quantile or a p-value is a look-up.
- `np.searchsorted(table, stat, side="right")` — how many sorted values are at or below stat (binary search).
- `/ len(table)` — a count turned into a share: the p-value.

adf: lagged changes, chosen by AIC

MATH

$$\Delta y_t = \gamma\, y_{t-1} + \Sigma_{i=1..p}\, b_i\, \Delta y_{t-i} + c + e_t; \quad AIC = m\, log(SSR / m) + 2k$$

CODE

```
def adf(y, regression="c", maxlag=None):
    y = np.asarray(y, float); n = len(y); k0 = 1 if regression == "c" else 2
    if maxlag is None:
        maxlag = min(n // 2 - k0 - 1, int(np.ceil(12 * (n / 100) ** 0.25)))
    dy = np.diff(y)
    def fit(p, start):
        r = np.arange(start, len(dy))
        det = [np.ones(len(r))] + ([np.arange(1, len(r) + 1)] if regression == "ct" else [])
        X = np.column_stack([y[r]] + [dy[r - i] for i in range(1, p + 1)] + det)
        b, ssr = np.linalg.lstsq(X, dy[r], rcond=None)[:2]; ssr = float(ssr[0]) if len(ssr) else
            float(np.sum((dy[r] - X @ b) ** 2))
        return b, ssr, X, len(r)
    aic = []
    for p in range(maxlag + 1):
        _, ssr, X, m = fit(p, maxlag); aic.append((m * np.log(ssr / m) + 2 * X.shape[1], p))
    p = min(aic)[1]
    b, ssr, X, m = fit(p, p)
    se = np.sqrt(ssr / (m - X.shape[1]) * np.linalg.inv(X.T @ X)[0, 0])
    return b[0] / se, p, m
```

- `def fit(p, start):` — a function defined inside a function; it can read y, dy and regression from the outer one.
- `[dy[r - i] for i in range(1, p + 1)]` — the p earlier changes, as columns: r - i shifts every row index back by i.
- `np.linalg.lstsq(X, dy[r], rcond=None)[:2]` — the coefficients and the sum of squared residuals; [:2] keeps those two.
- `aic.append((..., p)); min(aic)[1]` — tuples compare by their first item, then the second, so a tie goes to the smaller p.
- `fit(p, maxlag) then fit(p, p)` — every lag is compared on the same rows; the chosen one is refitted on all rows it allows.
- `np.linalg.inv(X.T @ X)[0, 0]` — the level coefficient's entry of $(X^TX)^{-1}$, for its standard error.

kpss: the opposite question

MATH

$$KPSS = \Sigma_t\, S_t^2 / (n^2\, s^2), \quad S_t = e_1 + ... + e_t, \quad s^2 = long\text{-}run\ variance$$

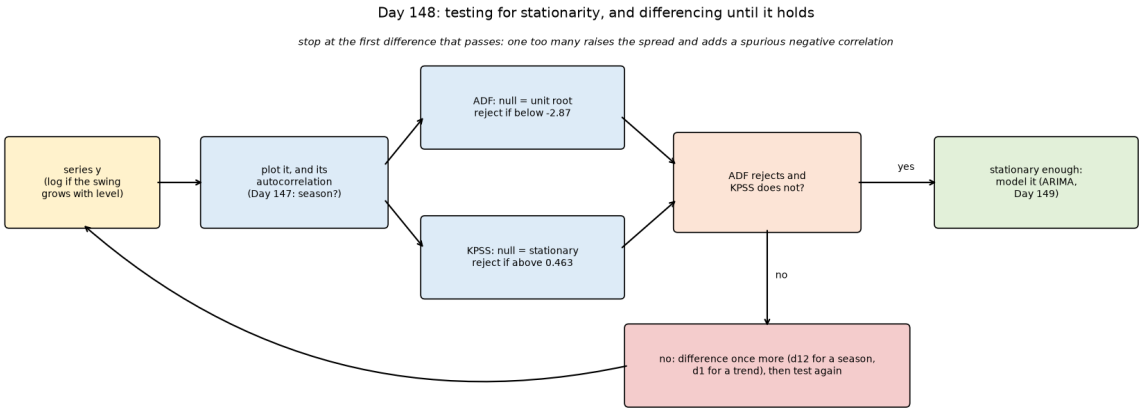
CODE

```
def kpss(y, regression="c", nlags=None):
    y = np.asarray(y, float); n = len(y)
    if nlags is None:
        nlags = min(int(np.ceil(12 * (n / 100) ** 0.25)), n - 1)
    if regression == "c":
        e = y - y.mean()
    else:
        X = np.column_stack([np.ones(n), np.arange(1, n + 1)]); e = y - X @ np.linalg.lstsq(X, y,
            rcond=None)[0]
    s2 = (np.sum(e * e) + 2 * sum((1 - k / (nlags + 1)) * np.sum(e[k:] * e[:-k]) for k in range(1,
        nlags + 1))) / n
    return float(np.sum(np.cumsum(e) ** 2) / n ** 2 / s2)
```

- `int(np.ceil(12 * (n / 100) ** 0.25))` — the lag rule shared with ADF's maximum; ceil rounds up.

- `sum((1 - k / (nlags + 1)) * ... for k in ...)` — a generator inside Python's sum: Bartlett weights falling from 1 towards 0.
- `np.cumsum(e)` — running totals of the deviations; for a stationary series they stay small.
- `float(...)` — a plain Python number rather than a numpy scalar.

SYSTEM DIAGRAM



Test, difference, test again.

Segment 2 — The experiments

simulate: four series with known behaviour

MATH

AR(1): $y_t = \phi\, y_{t-1} + e_t$; $\phi = 1$ is a random walk

CODE

```
def simulate(n=500, seed=0):
    rng = np.random.RandomState(seed); e = rng.standard_normal((4, n))
    ar = np.zeros(n)
    for t in range(1, n):
        ar[t] = 0.9 * ar[t - 1] + e[1, t]
    return {"white noise": e[0], "AR(1), phi 0.9": ar, "random walk": np.cumsum(e[2]),
            "trend + noise": 0.02 * np.arange(n) + e[3]}
```

- `e = rng.standard_normal((4, n))` — four rows of noise; e[0] is the first row.
- `ar[t] = 0.9 * ar[t - 1] + e[1, t]` — the AR(1) recursion, one step at a time: each value depends on the last.
- `return {"white noise": ..., ...}` — a dictionary keeps the series with their names, in order.

demo_meaning: the two halves

CODE

```
for name, y in list(sims.items()) + [("CO2", yc), ("log orders", np.log(ye))]:
    a, b = np.array_split(y, 2); r = acf(y, 12)
    print(f"{name:>22} {a.mean():15.3f} {b.mean():9.3f} {a.std():14.3f} {b.std():9.3f}
        {r[1]:10.3f} {r[12]:7.3f}")
```

- `list(sims.items()) + [("CO2", yc), ...]` — the dictionary's (name, series) pairs, with two more pairs added on.

- `a, b = np.array_split(y, 2)` — two halves, unpacked into two names (works for odd lengths too).
- `{a.mean():15.3f}` — a width of 15 characters and 3 decimals, so the columns line up.

demo_table: simulated against published

CODE

```
for reg, name in (("c", "constant"), ("ct", "constant + trend")):
    T = df_table(500, reg); tables[reg] = T; q = np.quantile(T, [0.01, 0.05, 0.10])
    m = mackinnoncrit(N=1, regression=reg, nobs=499)
    print(f"{name + ', simulated':>28} {q[0]:8.3f} {q[1]:8.3f} {q[2]:8.3f}   {np.mean(T <
        -1.645):.3f}")
    print(f"{name + ', MacKinnon':>28} {m[0]:8.3f} {m[1]:8.3f} {m[2]:8.3f}")
```

- `np.quantile(T, [0.01, 0.05, 0.10])` — three quantiles at once: the 1%, 5% and 10% critical values.
- `mackinnoncrit(N=1, regression=reg, nobs=499)` — statsmodels' published critical values, for the check.
- `np.mean(T < -1.645)` — the mean of True/False values is the share that are True.

demo_adf: the real series and their differences

CODE

```
for name, y in (("CO2", yc), ("CO2, d1", D(yc)), ("CO2, d12", D(yc, 12)), ("CO2, d1 d12", D(D(yc,
    12))),
                ("log orders", le), ("log orders, d1", D(le)), ("log orders, d12", D(le, 12)),
                    ("log orders, d1 d12", D(D(le, 12)))):
    s, p, m = adf(y); ref = adfuller(y, regression="c", autolag="AIC")
    worst = max(worst, abs(s - ref[0])); assert p == ref[2]
    pm = mc_pvalue(s, tab[500 if len(y) > 400 else 240]); r = acf(y, 12); keep[name] = (y, s)
    ps = f"{pm:.4f}" if pm > 0 else f"< {1 / SIMS:.4f}"
    print(f"{name:>20} {s:8.3f} {ref[0]:12.3f} {p:5d} {ps:>13} {ref[1]:13.4f} {r[1]:6.3f}
        {r[12]:7.3f} {y.std():8.4f}")
```

- `D = difference` — a function given a short second name, so D(D(yc, 12)) reads as d1 after d12.
- `assert p == ref[2]` — stops the run if our chosen lag ever differs from the library's.
- `f"{pm:.4f}" if pm > 0 else f"< {1 / SIMS:.4f}"` — a conditional expression picks how to print a p-value too small to resolve.

demo_kpss: reading the two tests together

CODE

```
a = adf(y)[0]; k = kpss(y); ref = sm_kpss(y, regression="c", nlags="legacy")[0]; worst =
    max(worst, abs(k - ref))
ra, rk = a < -2.87, k > 0.463
read = {(True, False): "both say stationary", (False, True): "both say not stationary",
        (True, True): "conflict: both reject", (False, False): "conflict: neither rejects"}[(ra,
            rk)]
print(f"{name:>20} {a:8.3f} {k:7.3f} {ref:12.3f}   {read}")
```

- `ra, rk = a < -2.87, k > 0.463` — two True/False answers in one line: does each test reject?
- `{(True, False): "...", ...}[(ra, rk)]` — a dictionary keyed by pairs, looked up at once: a compact four-way choice.
- `sm_kpss(y, regression="c", nlags="legacy")[0]` — the library's KPSS with the same lag rule; [0] is the statistic.

demo_power: many series at once

MATH

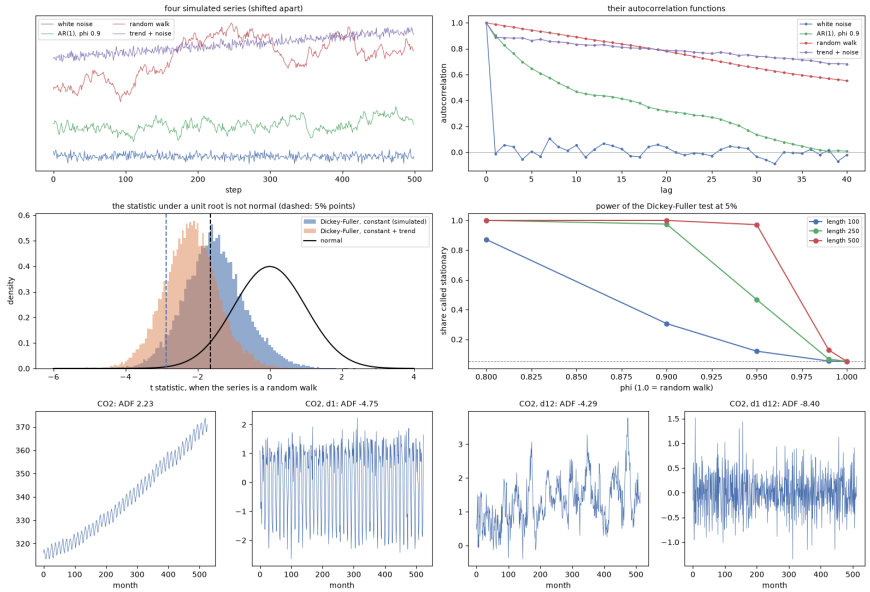
power = share of stationary series ($\phi < 1$) that the test calls stationary

CODE

```
for phi in phis:
    e = rng.standard_normal((n, POWER_SIMS)); Y = np.zeros((n, POWER_SIMS))
    Y[0] = e[0] / np.sqrt(1 - phi ** 2) if phi < 1 else 0.0
    for t in range(1, n):
        Y[t] = phi * Y[t - 1] + e[t]
    out.append(np.mean(df_stats(Y) < crit))
```

- `Y[0] = ... if phi < 1 else 0.0` — start at the long-run spread, 1 / sqrt(1 – ϕ^2); a walk has none, so it starts at 0.
- `Y[t] = phi * Y[t - 1] + e[t]` — one row at a time, but all 2,000 series in that row together.
- `df_stats(Y) < crit` — one statistic per column, compared with this length's simulated 5% point.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `import x as y / from m import f as g` — import under a different name.
- `contextlib.redirect_stdout(io.StringIO())` — send prints to a string instead of the screen.
- `np.diff(Y, axis=0)` — differences down each column.
- `np.linalg.qr(A)` — QR factorisation: Q has orthonormal columns.
- `np.linalg.lstsq(X, y, rcond=None)` — least squares: coefficients, residual sum, rank, singular values.
- `np.searchsorted(a, v, side="right")` — how many sorted values are at or below v.
- `np.quantile(x, [q1, q2])` — several quantiles at once.

End of Day 148 syntax reference. For the experiments and what they showed, see the Day 148 complete guide.